# Overall Detection

**Overall Detection**

1. Course Content
2. Case Demonstration
3. Source Code Analysis

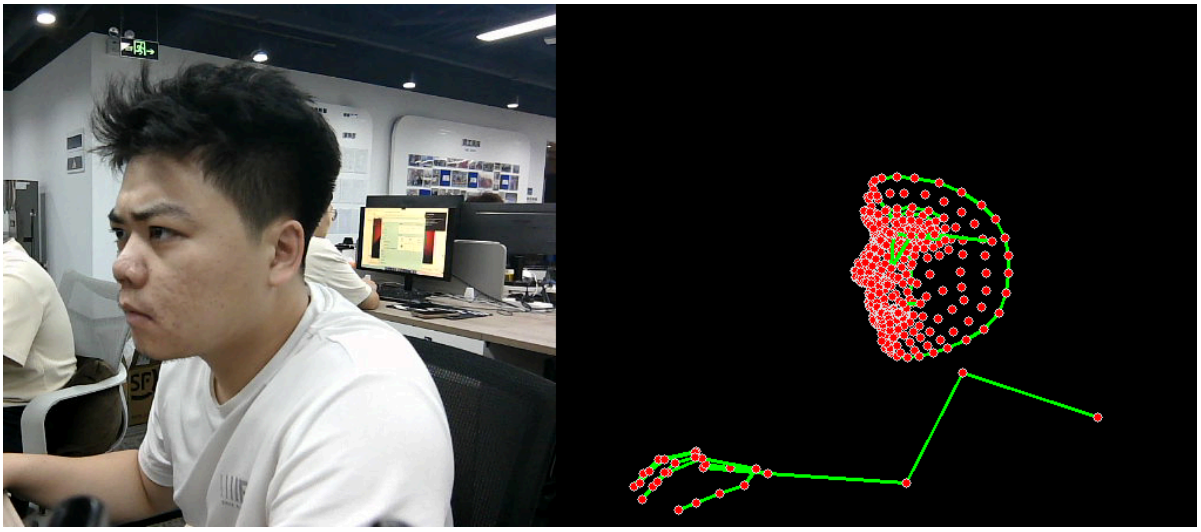
## 1. Course Content

- Run the example program to detect the whole body using the mediapipe framework, including the face, palms, and torso.

## 2. Case Demonstration

```
ros2 launch yahboomcar_bringup mediapipe_demo.launch.py demo:=Holistic
```

- After the program runs, as shown in the figure below, the detected overall points will be displayed on the right side of the image



## 3. Source Code Analysis

- Program code path:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/mediapipe/03_Holis
tic.py
```

- Import the required library files,

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Point
import mediapipe as mp
import cv2 as cv
import numpy as np
import time
import os
from cv_bridge import CvBridge
from sensor_msgs.msg import Image
from arm_msgs.msg import ArmJoints
import cv2
print("import done")
```

Initialize the data and define the Publisher and Subscriber,

```
    def __init__(self, name,staticMode=False, landmarks=True, detectionCon=0.5,
trackingCon=0.5):
        super().__init__(name)
        #Parameter declaration and retrieval

 self.declare_parameter('rgb_topic','/ascamera_hp60c/camera_publisher/rgb0/image'
)

 rgb_topic=self.get_parameter("rgb_topic").get_parameter_value().string_value
        self.mpHolistic = mp.solutions.holistic
        self.mpFaceMesh = mp.solutions.face_mesh
        self.mpHands = mp.solutions.hands
        self.mpPose = mp.solutions.pose
        self.mpDraw = mp.solutions.drawing_utils
        self.mpholistic = self.mpHolistic.Holistic(
            static_image_mode=staticMode,
            smooth_landmarks=landmarks,
            min_detection_confidence=detectionCon,
            min_tracking_confidence=trackingCon)
        self.lmDrawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 0,
255), thickness=-1, circle_radius=3)
        self.drawSpec = mp.solutions.drawing_utils.DrawingSpec(color=(0, 255, 0),
thickness=2, circle_radius=2)
        #create a publisher
        self.rgb_bridge = CvBridge()
```

Color image callback function,

```
def get_RGBImageCallBack(self,msg):
    #Use CvBridge to convert the color image message data into image data
    rgb_image = self.rgb_bridge.imgmsg_to_cv2(msg, "bgr8")
    #Pass the obtained image into the defined pubPosePoint function; draw=False
means not drawing the joint points on the original color image
    frame, img = self.findHolistic(rgb_image, draw=False)
    #Merge the two images
    dist = self.frame_combine(frame, img)
    key = cv2.waitKey(1)
    cv.imshow('dist', dist)
```

findHolistic function,

```
def findHolistic(self, frame, draw=True):
    #Create a new image based on the size of the input image; the data type of
the image is uint8
    img = np.zeros(frame.shape, np.uint8)
    #Convert the color space of the input image from BGR to RGB for subsequent
image processing
    img_RGB = cv.cvtColor(frame, cv.COLOR_BGR2RGB)
    #Call the process function in the mediapipe library for image processing;
during init, the self.mpholistic object was created and initialized
    self.results = self.mpholistic.process(img_RGB)
    #Determine whether a face was detected; if so, connect each point on the
blank image created earlier
    if self.results.face_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame, self.results.face_landmarks,
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.face_landmarks,
self.mpFaceMesh.FACEMESH_CONTOURS, self.lmDrawSpec, self.drawSpec)
    #Determine whether a torso was detected; if so, connect each point on the
blank image created earlier
    if self.results.pose_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.pose_landmarks,
self.mpPose.POSE_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    #Determine whether a left hand was detected; if so, connect each point on the
blank image created earlier
    if self.results.left_hand_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame,
self.results.left_hand_landmarks, self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec,
self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.left_hand_landmarks,
self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    #Determine whether a right hand was detected; if so, connect each point on
the blank image created earlier
    if self.results.right_hand_landmarks:
        if draw: self.mpDraw.draw_landmarks(frame,
self.results.right_hand_landmarks, self.mpHands.HAND_CONNECTIONS,
self.lmDrawSpec, self.drawSpec)
        self.mpDraw.draw_landmarks(img, self.results.right_hand_landmarks,
self.mpHands.HAND_CONNECTIONS, self.lmDrawSpec, self.drawSpec)
    return frame, img
```

The frame_combine image merging function was mentioned in the first lesson of this chapter; refer to the analysis of this function in "1. Hand Detection".